MicroFlat API — Frontend Integration Guide

Oct 22, 2026 · @Amit

1. Setup

All calls go to **https://api.microflat.ashtamtechnologies.com**; every path in this guide starts with `/api/...` and is appended to it, e.g. `POST https://api.microflat.ashtamtechnologies.com/api/auth/login`.

Base URLs

Environment	API base URL	Swagger
Server (use this)	`https://api.microflat.ashtamtechnologies.com`	api.microflat.ashtamtechnologies.com/swagger/index.html
Local (http profile)	`http://localhost:5115`	`http://localhost:5115/swagger`
Local (https profile)	`https://localhost:7182`	`https://localhost:7182/swagger`

Put the base URL in the frontend env files (Vite example). Do not include `/swagger` or `/api`.

```
# .env.production
VITE_API_BASE_URL=https://api.microflat.ashtamtechnologies.com

# .env.development (only when running the API locally)
VITE_API_BASE_URL=http://localhost:5115
```

CORS

The API accepts browser calls only from origins listed in `CorsSettings:AllowedOrigins` (backend appsettings). A new frontend host or port must be added by the backend team, otherwise the browser shows a CORS error.

Environment	Allowed origins today
Production	`https://web.microflat.ashtamtechnologies.com`

Environment	Allowed origins today
Development	`http://localhost:5173`, `http://localhost:3000`

Conventions

Topic	Rule
Format	JSON, camelCase property names
Auth header	`Authorization: Bearer <accessToken>` on every call except the public auth endpoints
Date-time	UTC ISO-8601 with `Z`, e.g. `2026-10-22T06:07:40Z`; convert to local time for display
Date only	`yyyy-MM-dd`, e.g. `effectiveDate`, `inactiveDate`
Paging	Query `pageNumber` (from 1) and `pageSize` (max 100, default 20); response `{ items, pageNumber, pageSize, totalCount, totalPages }`
IDs	Integers: `id` in most responses; some vendor responses use `vendorId` / `vendorDetailId`
Deletes	Mostly soft (status toggle); hard deletes are called out explicitly

2. Authentication & session handling

Log in once, send the access token on every call, refresh it on 401, and force a password change when `mustChangePassword` is true.

Token lifetimes

Token	Lifetime	Use
`accessToken` (JWT)	15 minutes	Send in the `Authorization` header

Token	Lifetime	Use
`refreshToken`	7 days	Only for `POST /api/auth/refresh-token`; each refresh returns a new one and the old one stops working

Keep tokens in memory (or secure storage) and never log them.

Login

`POST /api/auth/login` with `{ "email": "...", "password": "..." }`. Response `200`:

```
{
  "tokenType": "Bearer",
  "accessToken": "eyJhbGciOi...",
  "accessTokenExpiresOn": "2026-10-22T06:22:40Z",
  "refreshToken": "q7Z...",
  "refreshTokenExpiresOn": "2026-10-29T06:07:40Z",
  "user": {
    "userId": 2,
    "firstName": "Janvi",
    "lastName": "Motwani",
    "email": "janvi.motwani@ashtamtechnologies.com",
    "primaryRole": "Super Admin",
    "roles": ["Super Admin"],
    "mustChangePassword": true
  }
}
```

Errors: `401` "Invalid email or password." (same for unknown e-mail, wrong password or locked account), `401` account inactive, `401` temporary password expired, `429` too many attempts.

First login: forced password change

New users get a temporary password by e-mail, valid 72 hours.

1. If `user.mustChangePassword` is true, go straight to the Change Password screen.
2. Until it is changed, every API returns `403` with `code: "PASSWORD_CHANGE_REQUIRED"`, except `PUT /api/auth/change-password`, `POST /api/auth/logout`, `POST /api/auth/logout-all`, `GET /api/auth/me` and `GET /api/users/me`.

3. Call `PUT /api/auth/change-password` with `currentPassword` = the temporary password. The block lifts immediately; the same token keeps working.

Also handle it globally: any `403` whose body has `code === "PASSWORD_CHANGE_REQUIRED"` redirects to Change Password.

Refreshing the access token

On `401`, call `POST /api/auth/refresh-token` with `{ "refreshToken": "<current>" }`. The response has the login shape: replace both tokens, then retry the original request. If refresh returns `401`, clear tokens and go to Login.

Run only one refresh at a time and queue other requests meanwhile. Re-sending an already-used refresh token is treated as theft and ends the session.

Logout

Action	API
Logout this device	`POST /api/auth/logout` (no body)
Logout all devices	`POST /api/auth/logout-all`

The access token stops working immediately; clear tokens locally.

Forgot / reset password (public)

Screen	API	Body
Forgot password	`POST /api/auth/forgot-password`	`{ "email" }`; always `200` with the same message
Reset password page	`POST /api/auth/reset-password`	`{ "email", "token", "newPassword", "confirmPassword" }`

The e-mail link opens `<frontend>/reset-password?token=...&email=...`, so the frontend needs that route and must read both query parameters. The link expires in 30 minutes and works once; invalid or expired returns `400`. Passwords are 10-128 characters and must differ from the e-mail.

Axios setup (example)

```
import axios from "axios";

export const api = axios.create({ baseURL: import.meta.env.VITE_API_BASE_URL
});

let accessToken = "";
let refreshToken = "";
let refreshing: Promise<void> | null = null;

export function setTokens(a: string, r: string) { accessToken = a;
refreshToken = r; }

api.interceptors.request.use(cfg => {
  if (accessToken) cfg.headers.Authorization = `Bearer ${accessToken}`;
  return cfg;
});

api.interceptors.response.use(undefined, async error => {
  const { response, config } = error;
  if (response?.status === 403 && response.data?.code ===
"PASSWORD_CHANGE_REQUIRED") {
    window.location.href = "/change-password";
    return Promise.reject(error);
  }
  if (response?.status === 401 && refreshToken && !config._retried &&
!config.url.includes("/api/auth/")) {
    config._retried = true;
    refreshing ??= axios
      .post(`${import.meta.env.VITE_API_BASE_URL}/api/auth/refresh-token`, {
refreshToken })
      .then(r => setTokens(r.data.accessToken, r.data.refreshToken))
      .finally(() => { refreshing = null; });
    try { await refreshing; } catch { setTokens("", ""); window.location.href
= "/login"; return Promise.reject(error); }
    return api(config);
  }
  return Promise.reject(error);
});
```

3. Errors

Every error is a ProblemDetails object; show its `detail` to the user, it is always a safe, readable message.

```
{ "type": "...", "title": "Bad Request", "status": 400, "detail": "GstNo must be a valid 15-character GSTIN (e.g. 24AAACC1206D1ZM)." }
```

Field validation errors (`400`) list messages per field:

```
{ "title": "One or more validation errors occurred.", "status": 400,
  "errors": { "Email": ["The Email field is not a valid e-mail address."] } }
```

Status	Meaning	UI action
400	Invalid input or business rule	Show `detail` , or the `errors` per field
401	Not logged in / token expired	Refresh the token, else go to Login
403	No permission, or `PASSWORD_CHANGE_REQUIRED`	Show "not allowed", or redirect to Change Password
404	Record not found	Show not found
409	Duplicate, or someone else changed the record first	Show `detail` and reload the record
429	Too many requests (login / password endpoints)	"Try again in a minute"
500	Server error	Generic error message

4. Roles & permissions

"Admin" here means role **Super Admin** or **Admin**; read roles from `user.roles` (login or `GET /api/auth/me`) and hide what the user cannot do.

Area	Any logged-in user	Admin only
Own profile, change password	Yes	

Area	Any logged-in user	Admin only
Users & Roles	View role list	Users list / edit / status / unlock / roles, create users, role create / edit / status
Configuration (modules, document types, approval types)	View	Create, edit, activate / deactivate
Vendors	List, view, create, edit Draft / Rejected, documents, submit, start a change, add / edit bank details, upload bank documents	Rename, activate / deactivate, delete vendor; activate / deactivate bank account
Approvals	See own requests, cancel own pending request	See all, verify documents, approve, reject
Process flow diagram	View	Maintain flows, steps, arrows

Rules the API also enforces:

- A user cannot approve or reject a request they raised (Super Admin excepted): hide Approve / Reject when `request.initiatedBy` is the current user.
- Users cannot change their own roles or status; only a Super Admin can grant or remove "Super Admin".

5. Screen to API map

Each row is one screen action: the call to make and what to send. Paths are relative to the base URL.

5.1 Login & account

Screen / action	Method & URL	Notes
Login	`POST /api/auth/login`	Section 2

Screen / action	Method & URL	Notes
Change password (first login or menu)	`PUT /api/auth/change-password`	`{ currentPassword, newPassword, confirmPassword }`; other sessions are logged out
Forgot password	`POST /api/auth/forgot-password`	Section 2
Reset password page	`POST /api/auth/reset-password`	Section 2
App start / header user menu	`GET /api/auth/me`	User, roles, current session info
My profile (view)	`GET /api/users/me`	
My profile (save)	`PUT /api/users/me`	`{ firstName, lastName, mobile, secondaryPhone?, address? }`; e-mail and roles not editable here
Logout this device	`POST /api/auth/logout`	
Logout all devices	`POST /api/auth/logout-all`	
Refresh token	`POST /api/auth/refresh-token`	Section 2

5.2 Users & Roles (Admin)

Screen / action	Method & URL	Body / query
Users list	`GET /api/users`	`?search=&isActive=&roleId=&pageNumber=1&pageSize=20`
Create user (UserFormModal)	`POST /api/auth/register`	`{ firstName, lastName, email, mobile, secondaryPhone?, address?, roleIds: [..], primaryRoleId }`; no password, a temporary one is e-mailed; response `{ user, temporaryPasswordEmailSent, temporaryPasswordExpiresOn, message }`

Screen / action	Method & URL	Body / query
View user	`GET /api/users/{id}`	Includes `roles`, `isLockedOut`, `mustChangePassword`
Session log (UserViewPage)	`GET /api/users/{id}/sessions?take=50`	Login history
Edit user	`PUT /api/users/{id}`	`{ firstName, lastName, email, mobile, secondaryPhone?, address? }`
Activate / deactivate	`PATCH /api/users/{id}/status`	`{ "isActive": false }`; deactivating logs the user out everywhere
Unlock (after 5 wrong passwords)	`POST /api/users/{id}/unlock`	
Re-send temporary password	`POST /api/users/{id}/temporary-password`	When the first one expired or was lost
User roles (view)	`GET /api/users/{id}/roles`	
User roles (save)	`PUT /api/users/{id}/roles`	`{ roleIds: [11, 14], primaryRoleId: 11 }`
Roles list / dropdown	`GET /api/roles?isActive=true`	Any logged-in user
Role view	`GET /api/roles/{id}`	
Role create	`POST /api/roles`	`{ roleName }`
Role rename	`PUT /api/roles/{id}`	`{ roleName }`
Role activate / deactivate	`PATCH /api/roles/{id}/status`	`{ isActive }`; "Super Admin" and "Admin" cannot be renamed or deactivated

The very first user of an empty system is created with `POST /api/auth/register` without a token and becomes Super Admin.

5.3 Configuration masters

Screen / action	Method & URL	Body / query
Modules list	`GET /api/modules?isActive=`	
Module view	`GET /api/modules/{id}`	

Screen / action	Method & URL	Body / query
Module create	`POST /api/modules`	`{ moduleCode, moduleName }`
Module rename	`PUT /api/modules/{id}`	`{ moduleName }`
Module status	`PATCH /api/modules/{id}/status`	`{ isActive }`; VENDOR cannot be deactivated
Document types (DocumentTypesPage)	`GET /api/document-types?moduleCode=VENDOR&isActive=true`	Items carry `maxFileSizeMB`, `allowedExtensions`, `allowedExtensionList` for the file picker
Document type view	`GET /api/document-types/{id}`	
Document type create	`POST /api/document-types`	`{ moduleId, documentTypeName, description?, maxFileSizeMB, allowedExtensions: ".pdf,.jpg" }`
Document type edit	`PUT /api/document-types/{id}`	Same body
Document type status	`PATCH /api/document-types/{id}/status`	`{ isActive }`
Approval types list	`GET /api/approval-types?moduleId=&isActive=`	
Approval type view	`GET /api/approval-types/{id}`	
Approval type create	`POST /api/approval-types`	`{ moduleId, typeCode, typeName, approvalPermissionKey? }`
Approval type edit	`PUT /api/approval-types/{id}`	`{ typeName, approvalPermissionKey? }`

Screen / action	Method & URL	Body / query
Approval type status	`PATCH /api/approval-types/{id}/status`	`{ isActive }`; VENDOR types cannot be deactivated

5.4 Vendors

A vendor is a header (`vendorCode`, `vendorName`) plus dated versions of its details; only Draft or Rejected versions can be changed.

- `vendorCode` is always generated by the API (`VEN-0001`, `VEN-0002` …); do not send it.
- Version `approvalStatus`: `Draft` > `Pending` (submitted) > `Approved` or `Rejected`.
- Version `versionState` (Effective Dates page): `Current`, `Scheduled` (approved, future date), `History`, `Draft`, `Pending`, `Rejected`.
- Vendor `approvalStatus` is `Approved` once any version was approved, otherwise the latest version's status; `hasPendingChange` is true when an approved vendor has a change in progress.

Screen / action	Method & URL	Notes
Dashboard cards	`GET /api/vendors/summary`	`{ total, active, inactive, draft, pendingApproval, rejected, pendingChanges }`
Form dropdowns	`GET /api/vendors/lookups`	`{ approvalStatuses, bankAccountTypes, documentTypes }`
Vendors list	`GET /api/vendors`	`?search=&approvalStatus=&isActive=&hasPendingChange=&pageNumber=&pageSize=`; search matches code, name, contact, phone, e-mail, GST, city
Create vendor	`POST /api/vendors`	Body below; `openVersion.vendorDetailId` in the response is the draft to upload documents to
Vendor view	`GET /api/vendors/{id}`	`currentDetail` (approved, effective today, with documents), `openVersion`, `versions`, `bankDetailCount`
Effective dates page	`GET /api/vendors/{id}/versions`	Timeline, newest first
Open a version	`GET /api/vendor-versions/{versionId}`	Details, `attachments`, `latestApprovalRequest` (reject reason in `comments`)
Edit Draft / Rejected version	`PUT /api/vendor-versions/{versionId}`	Same body as `detail` below; Rejected becomes Draft
Delete Draft / Rejected version	`DELETE /api/vendor-versions/{versionId}`	Not the vendor's only version

Screen / action	Method & URL	Notes
Submit for approval	`POST /api/vendor-versions/{versionId}/submit`	`{ comments? }`; Super Admin may send `{ autoApprove: true }`
Change an approved vendor	`POST /api/vendors/{id}/versions`	`detail` fields + `effectiveDate` (today or later) + `copyAttachments` (default true) + `submitForApproval`; one change at a time (`409`)
Rename vendor (Admin)	`PUT /api/vendors/{id}`	`{ vendorName }`
Activate / deactivate (Admin)	`PATCH /api/vendors/{id}/status`	Deactivate `{ isActive: false, inactiveComment, inactiveDate? }`; activate `{ isActive: true }`
Delete vendor (Admin)	`DELETE /api/vendors/{id}`	Only never-approved vendors with nothing pending; deletes their documents
Approval history tab	`GET /api/vendors/{id}/approval-history`	All requests for all versions
Version documents list	`GET /api/vendor-versions/{versionId}/attachments`	
Version document upload	`POST /api/vendor-versions/{versionId}/attachments`	Multipart, section 6
Version document download	`GET /api/vendor-attachments/{attachmentId}/download`	Section 6
Version document delete	`DELETE /api/vendor-attachments/{attachmentId}`	Draft / Rejected versions only

Create vendor body (required: `vendorName`, `contactPersonName`, `phoneNo`, `address1`, `countryId`, `city`; optional fields may be `null` or `""`):

```
{
  "vendorName": "Acme Industrial Supplies",
  "submitForApproval": false,
  "detail": {
    "effectiveDate": "2026-10-22",
    "contactPersonName": "Ravi Shah",
    "phoneNo": "9876543210",
    "alternatePhoneNo": null,
    "email": "ravi@acme.example",
    "website": "acme.example",
    "gstNo": "24AAACC1206D1ZM",
    "address1": "Plot 12, GIDC",
    "address2": null,
    "countryId": 1,
    "stateId": 7,
    "city": "Ahmedabad",
```

```
    "zipCode": "380001",
    "notes": null
  }
}
```

`gstNo` must be a valid GSTIN not used by another vendor (`409`); `effectiveDate` defaults to today. `countryId` / `stateId` are plain numbers for now, with no Country / State API yet.

Recommended create flow:

1. `POST /api/vendors` with `submitForApproval: false` ; keep `openVersion.vendorDetailId` .
2. Upload documents with `POST /api/vendor-versions/{versionId}/attachments` , one call per file.
3. `POST /api/vendor-versions/{versionId}/submit` .

Which buttons to show:

Button	Show when
Edit / Upload document / Delete document / Submit	Version `approvalStatus` is `Draft` or `Rejected`
Delete version	Version is Draft / Rejected and the vendor has more than one version
New change (create version)	Vendor `approvalStatus` is `Approved` and `hasPendingChange` is false
Withdraw	Version is `Pending` and the user raised it or is Admin; cancel `latestApprovalRequest.id` (5.6)
Activate / Deactivate (Admin)	Vendor has `currentDetail`
Delete vendor (Admin)	Vendor `approvalStatus` is `Draft` or `Rejected`
Show reject reason	Version is `Rejected` : `latestApprovalRequest.comments`

5.5 Bank details

Screen / action	Method & URL	Notes
Bank details list	`GET /api/vendors/{id}/bank-details`	Account numbers masked (`XXXXXXXXXX6789`)
Add bank account	`POST /api/vendors/{id}/bank-details`	Body below
View bank account	`GET /api/vendor-bank-details/{bankId}`	Full account number + `attachments`
Edit bank account	`PUT /api/vendor-bank-details/{bankId}`	Same body
Activate / deactivate (Admin)	`PATCH /api/vendor-bank-details/{bankId}/status`	`{ isActive }`
Bank documents list	`GET /api/vendor-bank-details/{bankId}/attachments`	
Bank document upload	`POST /api/vendor-bank-details/{bankId}/attachments`	Multipart, section 6; only while the account is active
Bank document download	`GET /api/vendor-bank-attachments/{attachmentId}/download`	Section 6
Bank document delete	`DELETE /api/vendor-bank-attachments/{attachmentId}`	

```
{
  "accountHolderName": "Acme Industrial Supplies",
  "bankName": "HDFC Bank",
  "branchName": "Ashram Road",
  "accountNumber": "50100123456789",
  "accountType": "Current",
  "ifscCode": "HDFC0001234",
  "micrCode": "380240002",
  "iban": null,
  "swiftCode": null,
  "branchAddress": null,
  "remarks": null
}
```

`accountType` is `Current`, `Savings`, `Cash Credit` or `Overdraft`. Formats are checked (IFSC like `HDFC0001234`, MICR 9 digits, SWIFT 8 / 11 characters, IBAN); the same account number twice for a vendor returns `409`.

5.6 Approvals

Screen / action	Method & URL	Notes
Inbox	`GET /api/approval-requests`	`?status=Pending&moduleCode=VENDOR&approvalTypeId=&referenceId=&mine=&pageNumber=&pageSize=`; Admin sees all, others only their own; items carry `referenceTitle`, `typeName`, `initiatedByName`, `isDocumentVerified`
Counters / badges	`GET /api/approval-requests/summary?mine=false`	`{ pending, approved, rejected, cancelled }`
Request detail	`GET /api/approval-requests/{id}`	`{ request, vendorDetail }`; `vendorDetail` holds the submitted data and its documents
Verify documents (Admin)	`POST /api/approval-requests/{id}/verify-documents`	`{ comments? }`
Approve (Admin)	`POST /api/approval-requests/{id}/approve`	`{ comments?, isDocumentVerified: true, documentVerificationComments? }`; unverified returns `400`
Reject (Admin)	`POST /api/approval-requests/{id}/reject`	`{ comments }` (reason required); version becomes `Rejected`

Screen / action	Method & URL	Notes
Cancel / withdraw	`POST /api/approval-requests/{id}/cancel`	`{ comments? }`; requester or Admin; version returns to `Draft`

Approval `status` is `Pending`, `Approved`, `Rejected` or `Cancelled`; `typeCode` is `VENDOR_REGISTRATION` (new vendor) or `VENDOR_UPDATE` (change). Deciding an already-decided request returns `409`: reload.

5.7 Process flow diagrams

Screen / action	Method & URL	Notes
Show diagram for a module	`GET /api/process-flow/{moduleCode}`	e.g. `SALES`, `VENDOR`; display only
Admin: list flows	`GET /api/process-flow-admin`	Includes inactive
Admin: flow editor	`GET /api/process-flow-admin/{flowId}`	Includes inactive steps and arrows
Admin: create flow	`POST /api/process-flow-admin`	`{ moduleCode, name }`
Admin: update flow	`PUT /api/process-flow-admin/{flowId}`	`{ name, isActive }`
Admin: add step	`POST /api/process-flow-admin/{flowId}/steps`	`{ title, description?, iconKey?, colNo, rowNo, outcomeType }`
Admin: edit step	`PUT /api/process-flow-admin/steps/{stepId}`	Same body + `isActive`
Admin: remove step	`DELETE /api/process-flow-admin/steps/{stepId}`	Soft delete; its arrows are removed too
Admin: add arrow	`POST /api/process-flow-admin/{flowId}/transitions`	`{ fromStepId, toStepId, label?, outcomeType }`
Admin: edit arrow	`PUT /api/process-flow-admin/transitions/{id}`	Same body + `isActive`

Screen / action	Method & URL	Notes
Admin: remove arrow	`DELETE /api/process-flow-admin/transitions/{id}`	Soft delete

```
{
  "name": "Vendor Registration Flow",
  "steps": [ { "id": 9, "title": "Registration", "description": "...",
"iconKey": "user-plus", "colNo": 1, "rowNo": 1, "outcomeType": "Normal" } ],
  "transitions": [ { "fromStepId": 11, "toStepId": 12, "label": "Approved",
"outcomeType": "Success" } ]
}
```

Draw steps on a grid: `colNo` left to right, `rowNo` the lane. `iconKey` is a lucide icon name; `outcomeType` sets the colour (`Normal` neutral, `Success` green, `Rejected` red, `Warning` amber). Arrows may point backwards (loops).

6. File upload & download

Upload one file per multipart request; download with the token as a blob, because files are never public.

Upload (multipart/form-data)

Fields: `DocumentTypeId` (number), `DocumentName` (optional title), `File` (the file).

```
const form = new FormData();
form.append("DocumentTypeId", String(documentTypeId));
form.append("DocumentName", "GST Certificate 2026");   // optional
form.append("File", file);                             // File from <input
type="file">
await api.post(`/api/vendor-versions/${versionId}/attachments`, form); //
axios sets the multipart header
```

Before uploading, check on the client with the document type from `GET /api/vendors/lookups` : the extension is in `allowedExtensionList` (e.g. `.pdf, .jpg, .jpeg, .png`) and the size is at most `maxFileSizeMB` (default 5 MB). The API checks both again and also that the file really is that type; a renamed file returns `400` . Response `201` is the attachment (`id` , `fileName` , `fileSizeMB` , `downloadUrl` …).

Download / preview

A plain `<a href>` or `<img src>` gets `401`: fetch with the token and use a blob URL.

```
async function downloadAttachment(downloadUrl: string, fileName: string) {
  const res = await api.get(downloadUrl, { responseType: "blob" }); // e.g.
/api/vendor-attachments/5/download
  const url = URL.createObjectURL(res.data);
  const a = document.createElement("a");
  a.href = url; a.download = fileName; a.click();
  URL.revokeObjectURL(url);
}
```

For an in-page preview (PDF or image), set the blob URL as the `src` of an `<iframe>` or `<img>` instead of clicking a link.

7. Fixed values (dropdowns)

Load lists from the API rather than hard-coding ids; admins can add document types and roles.

Field	Values	Source
Vendor `approvalStatus`	Draft, Pending, Approved, Rejected	`GET /api/vendors/lookups`
Bank `accountType`	Current, Savings, Cash Credit, Overdraft	`GET /api/vendors/lookups`
Vendor document types	GST Certificate, PAN Card, MSME / Udyam Certificate, ISO Certificate, Bank Cancelled Cheque, Bank Passbook Copy, Bank Confirmation Letter, Bank Mandate Form, Other Document	`GET /api/vendors/lookups` (`documentTypes`)
Approval `status`	Pending, Approved, Rejected, Cancelled	Fixed
Version `versionState`	Current, Scheduled, History, Draft, Pending, Rejected	Fixed

Field	Values	Source
Process flow `outcomeType`	Normal, Success, Rejected, Warning	Fixed
Roles	Super Admin, Admin, Contributor, Supervisor, Client, Regional Manager, Country Manager, Continent Manager, Sales rep, Marketing Manager, Manager, Inspector, Operator, Viewer	`GET /api/roles`

8. End-to-end examples

Three common journeys, call by call (ids are examples).

New vendor until approved

#	Who	Call
1	Purchase user	`POST /api/vendors`: version id 12 (Draft)
2	Purchase user	`POST /api/vendor-versions/12/attachments`, once per document
3	Purchase user	`POST /api/vendor-versions/12/submit`: Pending, approval request 30
4	Admin	`GET /api/approval-requests?status=Pending`, then `GET /api/approval-requests/30` to review data and documents
5a	Admin	`POST /api/approval-requests/30/approve` with `{ "isDocumentVerified": true }`: vendor Approved
5b	Admin	Or `POST /api/approval-requests/30/reject` with `{ "comments": "PAN copy unreadable" }`: the user edits with `PUT /api/vendor-versions/12`, fixes documents and submits again

Changing an approved vendor's address from next month

1. `POST /api/vendors/{id}/versions` with the new data and `"effectiveDate": "2026-11-01"` (documents are copied).
2. Upload or replace documents on the new version if needed.
3. `POST /api/vendor-versions/{newVersionId}/submit`; after Admin approval the version shows `Scheduled` until 1 Nov, then `Current`.

Creating a user

1. Admin calls `POST /api/auth/register` with roles; the user receives a temporary password by e-mail.
2. The user logs in, gets `mustChangePassword: true`, and changes it with `PUT /api/auth/change-password`.
3. If the e-mail was lost or expired, Admin calls `POST /api/users/{id}/temporary-password`.